MACHINE LEARNING LABORATORY - OBSERVATION

1. Experiment No.: 6	**Date:** 29/10/26

2. Experiment Question

Write a program to implement the Naive Bayes algorithm in Python and to display the results using confusion matrix and accuracy.

3. Aim

To implement the Gaussian Naive Bayes classifier in Python, to classify the Iris data set, and to evaluate it using a confusion matrix, accuracy, precision, recall and F1-score.

4. Concept

Naive Bayes is a probabilistic classifier based on Bayes' theorem. It computes the posterior probability of each class for a given sample and picks the class with the highest value. It is called naive because it assumes that the features are conditionally independent given the class, so the joint likelihood is just the product of the individual feature likelihoods.

For continuous features, the Gaussian Naive Bayes model assumes each feature follows a normal distribution within each class, and estimates the mean and variance of every feature for every class from the training data. In the program, log-probabilities are added instead of multiplying probabilities, to avoid numerical underflow.

A confusion matrix tabulates actual classes against predicted classes. The diagonal holds the correct predictions and the off-diagonal cells hold the misclassifications.

5. Dataset Description

The Iris data set (150 samples, 4 numeric features, 3 classes) is used with the same 70:30 stratified split as Experiment 5 (105 training and 45 testing samples).

Item	Details	Notes
Features	Sepal length, sepal width, petal length, petal width	Continuous, in centimetres
Classes	Setosa, Versicolor, Virginica	Prior of each class = 1/3
Testing set	15 samples of each class	45 in total

6. Algorithm

1. Training: for each class c, compute the prior P(c), and the mean and variance of every feature.
2. Testing: for each test sample x, compute the Gaussian likelihood of every feature for every class.
3. Compute the log-posterior of each class: log P(c) + SUM[log P(x_i | c)].
4. Assign the class with the highest log-posterior.
5. Build the confusion matrix from the actual and predicted labels.

6. Compute accuracy, precision, recall and F1-score from the matrix.

7. Step-by-step Procedure

1. Load the Iris data set and split it into training and testing sets.
2. Create the GaussianNB class: fit() estimates priors, means and variances; predict() returns the class with the highest log-posterior.
3. Train the model on the 105 training samples.
4. Predict the classes of the 45 test samples.
5. Compute and display the confusion matrix and the accuracy.
6. Compute the per-class precision, recall and F1-score and plot the confusion matrix.

8. Formulae

Bayes' theorem

$$P(c \mid x) = [\, P(x \mid c) * P(c) \,] / P(x)$$

Naive (independence) assumption

$$P(x \mid c) = P(x1 \mid c) * P(x2 \mid c) * \ldots * P(xn \mid c)$$

Gaussian likelihood

$$P(x_i \mid c) = [\, 1 / \mathrm{sqrt}(2 * \pi * var_c) \,] * \exp(-(x_i - mean_c)^2 / (2 * var_c))$$

Decision rule

$$\text{predicted class} = \text{argmax over } c \text{ of } [\, \log P(c) + \text{SUM} \log P(x_i \mid c) \,]$$

Evaluation measures

$$Accuracy = (TP + TN) / (TP + TN + FP + FN)$$

$$Precision = TP / (TP + FP), \quad Recall = TP / (TP + FN), \quad F1 = 2PR / (P + R)$$

9. Python Program

```
"""
Experiment 6: Naive Bayes classifier (Gaussian) with Confusion Matrix and Accuracy
Dataset : Iris (150 samples, 4 features, 3 classes)
"""
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split


class GaussianNB:
    def fit(self, X, y):
        self.classes = np.unique(y)
        self.mean = np.array([X[y == c].mean(axis=0) for c in self.classes])
        self.var = np.array([X[y == c].var(axis=0) for c in self.classes]) + 1e-9
        self.prior = np.array([np.mean(y == c) for c in self.classes])


    def _log_posterior(self, x):
        # log P(c) + sum_i log P(x_i | c), Gaussian likelihood
        log_like = -0.5 * np.log(2 * np.pi * self.var) - ((x - self.mean) ** 2) / (2 * self.var)
        return np.log(self.prior) + log_like.sum(axis=1)


    def predict(self, X):
        return np.array([self.classes[np.argmax(self._log_posterior(x))] for x in X])


def confusion_matrix(y_true, y_pred, n):
    cm = np.zeros((n, n), dtype=int)
    for t, p in zip(y_true, y_pred):
        cm[t, p] += 1
    return cm


iris = load_iris()
X_tr, X_te, y_tr, y_te = train_test_split(iris.data, iris.target, test_size=0.3,
                                          random_state=42, stratify=iris.target)
nb = GaussianNB(); nb.fit(X_tr, y_tr)
pred = nb.predict(X_te)


print("Class priors:", np.round(nb.prior, 3))
print("\nClass-wise feature means:")
for c, name in enumerate(iris.target_names):
    print(f"  {name:<11}", np.round(nb.mean[c], 3))


cm = confusion_matrix(y_te, pred, 3)
print("\nConfusion Matrix (rows = actual, columns = predicted):")
print(f"{'':<12}" + "".join(f"{n:>12}" for n in iris.target_names))
for i, n in enumerate(iris.target_names):
    print(f"{n:<12}" + "".join(f"{v:>12}" for v in cm[i]))


accuracy = np.trace(cm) / cm.sum()
print(f"\nAccuracy = {np.trace(cm)}/{cm.sum()} = {accuracy*100:.2f}%")
print("\nPer-class metrics:")
print(f"{'Class':<12}{'Precision':>10}{'Recall':>10}{'F1':>10}")
for i, n in enumerate(iris.target_names):
    p = cm[i, i] / max(cm[:, i].sum(), 1); r = cm[i, i] / max(cm[i].sum(), 1)
    f1 = 2 * p * r / (p + r) if p + r else 0
    print(f"{n:<12}{p:>10.3f}{r:>10.3f}{f1:>10.3f}")


# ---------------------------------------------------------------- plot
fig, ax = plt.subplots(figsize=(4.6, 4))
ax.imshow(cm, cmap="Greys", vmin=0, vmax=cm.max() * 1.6)
ax.set_xticks(range(3)); ax.set_yticks(range(3))
ax.set_xticklabels(iris.target_names, fontsize=8); ax.set_yticklabels(iris.target_names,
fontsize=8)
for i in range(3):
    for j in range(3):
        ax.text(j, i, cm[i, j], ha="center", va="center", fontsize=13,
                color="white" if cm[i, j] > cm.max() * 0.6 else "black")
ax.set_xlabel("Predicted class"); ax.set_ylabel("Actual class")
ax.set_title("Confusion matrix - Naive Bayes")
plt.tight_layout(); plt.savefig("figs/exp6_cm.png", dpi=200); plt.close()
```

10. Output

```
Class priors: [0.333 0.333 0.333]


Class-wise feature means:
  setosa      [4.989 3.426 1.486 0.24 ]
  versicolor  [5.949 2.731 4.237 1.309]
  virginica   [6.683 3.009 5.631 2.069]


Confusion Matrix (rows = actual, columns = predicted):
                 setosa  versicolor   virginica
setosa               15           0           0
versicolor            0          14           1
virginica             0           3          12


Accuracy = 41/45 = 91.11%


Per-class metrics:
Class        Precision    Recall        F1
setosa           1.000     1.000     1.000
versicolor       0.824     0.933     0.875
virginica        0.923     0.800     0.857
```

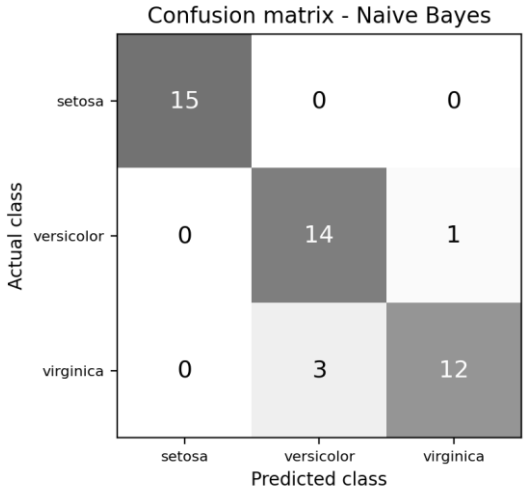


Fig. 6.1 Confusion matrix of the Naive Bayes classifier

11. Result / Conclusion

The Gaussian Naive Bayes classifier correctly classified 41 of the 45 test samples, giving an accuracy of 91.11%. All Setosa samples were classified correctly. The only confusion is between Versicolor and Virginica (4 samples), which overlap in feature space. Hence the Naive Bayes algorithm was implemented successfully and evaluated using the confusion matrix and accuracy.